

Class Inspector

GoJS® Diagramming Components
version 2.2.5
by Northwoods Software®

This is an extension and not part of the main GoJS library. Note that the API for this class may change at any time. If you intend to use an extension in production, you should copy the code to your own source directory. Extensions can be found in the GoJS kit under the `extensions`, `extensionsTS` (UMD modules), or `extensionsJSM` (ES6 modules) folders. See the [Extensions intro page](#) for more information.

Hierarchy

- **Inspector**

This class implements an inspector for GoJS model data objects. The constructor takes three arguments:

- `divId` **string** a string referencing the HTML ID of the to-be inspector's div
- `diagram` **Diagram** a reference to a GoJS Diagram
- `options` **Object** an optional JS Object describing options for the inspector

Options:

- `inspectSelection` **boolean** see [inspectSelection](#)
- `includesOwnProperties` **boolean** see [includesOwnProperties](#)
- `properties` **Object** see [properties](#)
- `propertyModified` **function(propertyName, newValue, inspector)** see [propertyModified](#)
- `multipleSelection` **boolean** see [multipleSelection](#)
- `showUnionProperties` **boolean** see [showUnionProperties](#)
- `showLimit` **number** see [showLimit](#)

Options for properties:

- `show` **boolean | function** a boolean value to show or hide the property from the inspector, or a predicate function to show conditionally.
- `readOnly` **boolean | function** whether or not the property is read-only
- `type` **string** a string describing the data **type**. Supported values: "string|number|boolean|color|arrayofnumber|point|rect|size|spot|margin|select|date|datetime-local|time"
- `defaultValue` **any** a default value for the property. Defaults to the empty string.
- `choices` **Array | function** when `type === "select"`, the Array of choices to use or a function that returns the Array of choices.

Example usage of Inspector:

```
var inspector = new Inspector("myInspector", myDiagram,
{
  includesOwnProperties: false,
  properties: {
    "key": { show: Inspector.showIfPresent, readOnly: true },
    "comments": { show: Inspector.showIfNode },
    "LinkComments": { show: Inspector.showIfLink },
    "chosen": { show: Inspector.showIfNode, type: "checkbox" },
    "state": { show: Inspector.showIfNode, type: "select", choices: ["Stopped", "Parked", "M
  }
});
```

This is the basic HTML Structure that the Inspector creates within the given DIV element:

```
<div id="divid" class="inspector">
  <tr>
    <td>propertyName</td>
    <td><input value=propertyValue /></td>
  </tr>
  ...
</div>
```

If you want to experiment with this extension, try the [Data Inspector](#) sample.

Index

Constructors

[constructor](#)

Properties

[diagram](#)
[div](#)
[includesOwnProperties](#)
[inspectSelection](#)
[inspectedObject](#)
[multipleSelection](#)
[properties](#)
[propertyModified](#)
[showLimit](#)
[showUnionProperties](#)

Methods

[buildPropertyRow](#)
[canEditProperty](#)
[canShowProperty](#)
[inspectObject](#)
[showIfGroup](#)
[showIfLink](#)
[showIfNode](#)
[showIfPresent](#)

Constructors

constructor

```
new Inspector(divid: string, diagram: Diagram, options?: {}): Inspector
```

Constructs an Inspector and sets up properties based on the options provided. Also sets up change listeners on the Diagram so the Inspector stays up-to-date.

Parameters

- **divid:** *string*
a string referencing the HTML ID of the to-be Inspector's div
- **diagram:** *Diagram*
a reference to a GoJS Diagram
- **Optional options:** {}
an optional JS Object describing options for the inspector
 - **[index:** *string*]: *any*

Returns *Inspector*

Properties

diagram : *Diagram*

Gets or sets the *Diagram* associated with this Inspector.

Read-only div : *HTMLDivElement*

This read-only property returns the `HTMLElement` containing the Inspector.

includesOwnProperties : *boolean*

Gets or sets whether the Inspector includes all properties currently on the inspected object.

The default value is true.

inspectSelection : *boolean*

Gets or sets whether the Inspector automatically inspects the associated Diagram's selection. When set to false, the Inspector won't show anything until [inspectObject](#) is called.

The default value is true.

Read-only **inspectedObject** : *ObjectData*

This read-only property returns the object currently being inspected.

To set the inspected object, call [inspectObject](#).

multipleSelection : *boolean*

Gets or sets whether the Inspector displays properties for multiple selected objects or just the first.

The default value is false, meaning only the first item in the [Diagram.selection](#) is inspected.

properties : *ObjectData*

Gets or sets the properties that the Inspector will inspect, maybe setting options for those properties. The object should contain string: Object pairs representing propertyName: propertyOptions. Can be used to include or exclude additional properties.

The default value is an empty object.

propertyModified : (**a**: *string*, **b**: *string*, **c**: *Inspector*) => *void*

Gets or sets the function to be called when a property is modified by the Inspector. The first parameter will be the property name, the second will be the new value, and the third will be a reference to this Inspector.

The default value is null, meaning nothing will be done.

showLimit : *number*

Gets or sets how many objects will be displayed when [multipleSelection](#) is true.

The default value is 0, meaning all selected objects will be displayed for a given property.

showUnionProperties : *boolean*

Gets or sets whether the Inspector displays the union or intersection of properties for multiple selected objects.

The default value is false, meaning the intersection of properties is inspected.

Methods

buildPropertyRow

`buildPropertyRow(propertyName: string, propertyValue: any): HTMLTableRowElement`

This sets `inspectedProperties[propertyName]` and creates the HTML table row for a given property:

```
<tr>
  <td>propertyName</td>
  <td><input value=propertyValue /></td>
</tr>
```

This method can be customized to change how an Inspector row is rendered.

Parameters

- **propertyName:** *string*
the property name
- **propertyValue:** *any*
the property value

Returns *HTMLTableRowElement*

the table row

canEditProperty

```
canEditProperty(propertyName: string, propertyDesc: ObjectData, inspectedObject:  
ObjectData): boolean
```

This predicate should be false if the given property should not be editable by the user. Normally it only checks the value of "readOnly" on the property descriptor.

The default value is true.

Parameters

- **propertyName:** *string*
the property name
- **propertyDesc:** *ObjectData*
the property descriptor
- **inspectedObject:** *ObjectData*
the data object

Returns *boolean*

whether a particular property should be shown in this Inspector

canShowProperty

```
canShowProperty(propertyName: string, propertyDesc: ObjectData, inspectedObject:  
ObjectData): boolean
```

This predicate should be false if the given property should not be shown. Normally it only checks the value of "show" on the property descriptor.

The default value is true.

Parameters

- **propertyName:** *string*
the property name
- **propertyDesc:** *ObjectData*
the property descriptor
- **inspectedObject:** *ObjectData*
the data object

Returns *boolean*

whether a particular property should be shown in this Inspector

inspectObject

```
inspectObject(object?: ObjectData): void
```

Update the HTML state of this Inspector with the given object.

If passed an object, the Inspector will inspect that object. If passed null, this will do nothing. If no parameter is supplied, the [inspectedObject](#) will be set based on the value of [inspectSelection](#).

Parameters

- **Optional** **object:** *ObjectData*
an optional argument, used when [inspectSelection](#) is false to set [inspectedObject](#) and show and edit that object's properties.

Returns *void*

Static showIfGroup

```
showIfGroup(part: Part): boolean
```

This predicate function can be used as a value for the [show](#) option for properties. When used, the property will only be shown when inspecting a [Group](#).

Parameters

- **part:** *Part*
the Part being inspected

Returns *boolean*

Static showIfLink

```
showIfLink(part: Part): boolean
```

This predicate function can be used as a value for the [show](#) option for properties. When used, the property will only be shown when inspecting a [Link](#).

Parameters

- **part:** *Part*
the Part being inspected

Returns *boolean*

Static **showIfNode**

```
showIfNode(part: Part): boolean
```

This predicate function can be used as a value for the **show** option for properties. When used, the property will only be shown when inspecting a *Node*.

Parameters

- **part:** *Part*
the Part being inspected

Returns *boolean*

Static **showIfPresent**

```
showIfPresent(data: Part, propname: string): boolean
```

This predicate function can be used as a value for the **show** option for properties. When used, the property will only be shown if present. Useful for properties such as **key**, which will be shown on Nodes and Groups, but normally not on Links

Parameters

- **data:** *Part*
- **propname:** *string*
the property to check presence of

Returns *boolean*

Copyright © 1998-2021 by Northwoods Software Corporation.